Hochschule: Hochschule Darmstadt

Fachbereich: Media

Studiengang: Information Science

Module: KI Anwendungsentwicklung

„Template Suggester“

Eingereicht am: 15.02.2026

Veranstaltungsart: Seminar

Dozent: Prof. Dr. Bernhard Thull

Vorgelegt von: Kuberlinov, Denis

Matrikelnummer: 1121541

Abstract

The Template Suggester is a feasibility prototype designed to assist in selecting suitable reply templates for recurring email inquiries. The core concept is sentence-embedded template matching: the system compares incoming emails against a library of pre-written templates to identify the most semantically similar response.

In the current iteration, templates are stored in a JSON structure containing titles, descriptions, keywords, and body text. The system encodes these templates into vector embeddings. When a user inputs an email, it is encoded into the same vector space, and the system calculates cosine similarity (implemented as a dot product after normalization) to rank the templates. The application then outputs the body of the best-matching template for the user to copy, edit, and send.

1. Motivation, Goals and project requirements

Motivation
As a Student Assistant (HiWi) at the university, I frequently address email inquiries requesting information that is already available on the website (e.g., application procedures). To manage this, our team uses an Outlook folder containing reusable templates in German and English. These templates typically provide brief explanations and links to detailed online resources.

However, this workflow presents challenges, particularly during the onboarding of new HiWis. The static folder structure can be confusing, and due to the similarity between certain topics, it is often difficult for new team members to determine if a relevant template exists or where to find it. This project aims to optimize this workflow by developing a tool that accepts a raw email as input and suggests the most appropriate template.

Goals
The primary objective was to build a prototype AI application to streamline the email response process and facilitate the onboarding of new staff.

Functional Goals:

- Develop a Command Line Interface (CLI) that accepts plain text email input.
- Implement a semantic search mechanism to classify the request type (e.g., "scholarships," "application process").
- Accurately select the most suitable template from a predefined JSON library.
- Display the suggested reply for human review and editing.

Learning Goals:

- Evaluate the technical viability of embedding-based retrieval for this specific use case.
- Gain practical experience implementing Transformer models with Python.
- Improve general Python development skills.

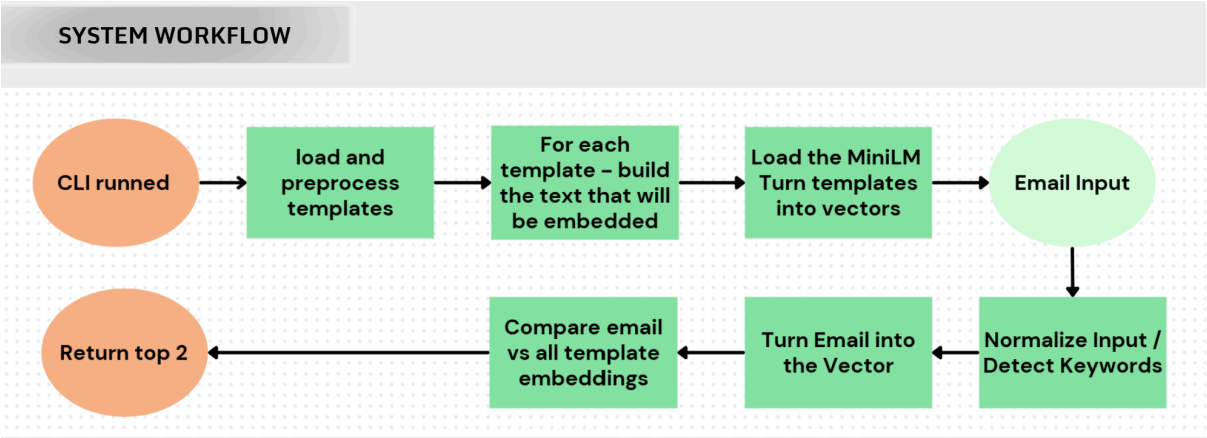
Requirements
A critical non-functional requirement was offline capability. Since incoming emails may contain sensitive personal data, the application must not send data to external APIs. The prototype ensures data privacy by running entirely locally, loading the model from a local path, and disabling external network calls during inference.

Tools and Technologies

The project utilizes a Python-based CLI architecture.

- **Core Language:** Python
- **Data Storage:** JSON (for the template structure).
- **NLP Engine:** sentence-transformer (all-MiniLM-L6-v2) for computing vector embeddings.
- **Scientific Computing:** numpy for dot-product calculations and matrix operations.

2. System Workflow



Startup (before any email is pasted):

1. Load templates from templates.json.
2. Select English fields (lang="en") and build a single doc_text per template by concatenating title, description, keywords, and body.
3. Load the local MiniLM model from the provided folder path and enforce offline mode.

4. Encode all template doc_text values once and store them as a matrix of template embeddings (cached in memory).

Per email (after the user pastes an email):

5. Read the pasted email until the user types END on its own line.
6. Encode the email into an embedding (normalized).
7. Compute similarity scores between the email embedding and all template embeddings (dot product).
8. Sort by similarity and print the top matches, then print the best template body separated by divider lines.

3. Implementation Details

The prototype is split into modular Python scripts to separate concerns between data processing, embedding logic, and user interaction.

1. Data & Preprocessing (preprocessing.py, templates.json)
The templates are stored in a structured JSON file. A custom function, load_templates_from_json, parses this file. To ensure the model "understands" the template context, I didn't just embed the template body. Instead, I constructed a composite "document text" for each template by concatenating:

Title + Description + Keywords + Body Text

This should ensure that even if the body text is generic, the semantic search has metadata to work with.

2. Embedding Pipeline (embeddings.py)
This module handles the core AI logic:

- **Loading:** It loads the local all-MiniLM-L6-v2 model.
- **Indexing:** On startup, it converts all template "document texts" into a matrix of embeddings.
- **Search:** When an email comes in, it is encoded into a vector. The system calculates the dot product between the email vector and the template matrix to find the nearest semantic neighbors.

3. CLI Interface (cli_embed_ver.py)
The user interface is a terminal script. It waits for the user to paste an email (supporting multi-line input) and marks the end of input with a specific keyword (END). It then displays the top 3 matching templates along with a confidence score and the full body text of the best match.

4. Current Status and Evaluation

The current prototype demonstrates a complete, functional offline workflow: it successfully loads templates, creates a vector index, encodes incoming emails, and ranks them by similarity to output a CLI response.

Evaluation
To assess the prototype's viability, I conducted a manual evaluation using 19 test emails. These emails were randomly selected from Outlook. Initially, there were 20 emails, but after a manual review, one email was identified as a special case that could not be answered with a template.

I compared the system's "Top-1" recommendation against a manual ground truth (the template a human would have actually chosen).

- Top-1 Accuracy: 12/19 (~63%)
- **Observation of "Score Clustering":** In many cases the similarity scores for the top 3 templates were extremely close (e.g., 0.563, 0.551, 0.544).

5. Conclusion and Limitations

The project confirms that offline sentence-embedded matching is a technically feasible architecture for a Template Suggester, but the current prototype requires significant refinement to be operationally reliable.

Critical Limitations

- **Score Clustering and Domain Specificity:** A recurring issue is that the Top-3 results often share extremely similar confidence scores (e.g., margins of only 0.01–0.05). This is likely due to domain-specific vector compression: since all templates relate to the same narrow context (university administration) and share boilerplate phrases (e.g., "Dear Student," "Application", "best regards"), they occupy a cramped region of the vector space. The general-purpose model struggles to distinguish between these subtle variations without additional signal.
- **Lack of "Uncertainty Handling":** The system currently lacks the logic to flag these ambiguous cases. It blindly presents the mathematical "winner" even when the margin of victory is negligible, which can lead to incorrect Top-1 suggestions.
- **Evaluation Constraints:** While 19 test emails were sufficient for a feasibility check, the sample size is too small to generalize performance or identify rare edge cases.

Final Verdict
With a Top-1 accuracy of 63% on the preliminary test set, the system is not yet

precise enough for autonomous selection. The "score clustering" phenomenon indicates that the current embedding strategy is effective for narrowing 20+ templates down to a relevant shortlist, but fails to provide the decisive separation needed for a "one-click" solution. To escape this limitation, implementation of hybrid scoring and/or aggressive data engineering (removing boilerplate text from the embedding input) is required.

Future Development / Next Steps

- **Prompt Engineering for Embeddings:** Improve separation between templates by removing boilerplate text (e.g., "Dear Student") from the embedding input and using keyword boosting (repeating high-value tokens like "Visa" or "Thesis") to artificially weight the vectors toward their specific intent.
- **Hybrid Search:** Implement a keyword-based scorer (TF-IDF) alongside the semantic model. This would help break ties in "score clusters" by favoring templates that contain the exact terminology used in the student's email.
- **Re-enable Placeholders:** Uncomment the placeholder extraction logic to provide users with a "checklist" of fields they need to fill in before sending the email.
- **Multi-language Support:** Add a toggle or simple language detection to switch the lang parameter, enabling the use of the German fields already present in the JSON.

6. Lessons Learned

Data Engineering: Context is the label on the book spine

- **What happened:** I initially considered embedding only the body text of the reply templates. However, as shown in embeddings.py, I ultimately decided to construct a composite doc_text by concatenating the title, description, keywords, and body.
- **What I learned:** Sentence embeddings rely heavily on information density. If a template body only contains generic text like "See the attached link," the model lacks the semantic context to classify it correctly. This is comparable to **finding a book in a library**: if a book has a blank cover, it is impossible to categorize. By injecting metadata (title/keywords) into the embedding string, I effectively placed a descriptive label on the book's spine, allowing the "librarian" (the model) to place it on the correct shelf.
- **What I'd do differently:** I would standardize the description field in the JSON schema to always include a specific "intent tag" (e.g., intent: rejection), ensuring that templates with similar phrasing but different outcomes are mathematically separated.

Dependency Management: Offline-first requires self-sufficiency

- **What happened:** To meet privacy requirements, I restricted the system from making external API calls by setting TRANSFORMERS_OFFLINE=1 in the loading script.
- **What I learned:** Developing an offline-first application changes the workflow significantly. If the specific model files are not physically present in the local models/ directory, the system fails immediately. Dependency management becomes much stricter when you remove the safety net of auto-downloading.
- **What I'd do differently:** I would implement a robust startup check that verifies the file integrity of the local model folder and provides a clear, human-readable error message instead of letting the library crash with a generic "file not found" exception.

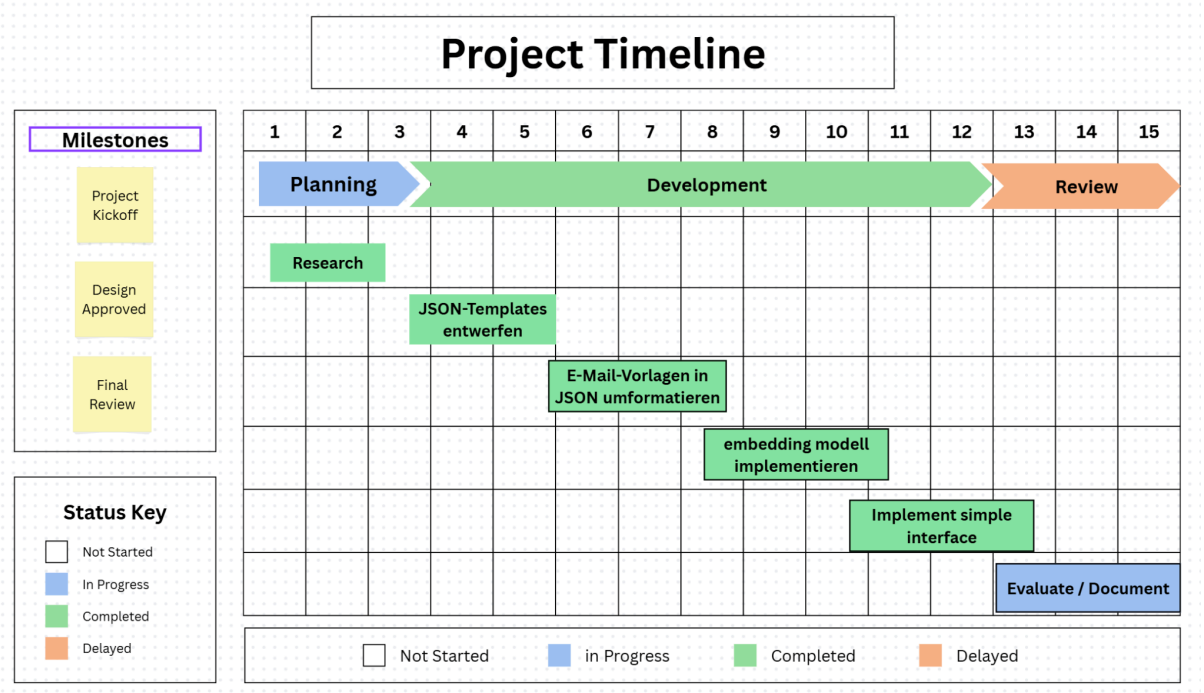
User Experience Ergonomics

- **What happened:** The current CLI interaction (cli_embed_ver.py) is technically functional but strict: it requires the user to paste their text and then manually type "END" on a new line to process the input.
- **What I learned:** Even if the underlying mathematics are correct, the interface defines the tool's usability. I found that pasting emails with headers often confused the output, and the rigid termination requirement felt outdated compared to modern interactions.
- **What I'd do differently:** I would implement a "loop mode" that keeps the model loaded and ready for the next query, removing the need to restart the script and reducing the friction for the user.

Scope Definition

- **What happened:** I wrote a extract_placeholders function in preprocessing.py to identify fields like {student_name}, but I commented out the logic that displays these placeholders in the final CLI output.
- **What I learned:** In a rapid prototype, it is tempting to leave "half-finished" code for later. The placeholders were a "nice-to-have" feature that I didn't have time to perfect, so they remained as dead code in the submission, blurring the definition of the prototype's scope.
- **What I'd do differently:** I would strictly define the Minimum Viable Product (MVP) earlier. If a feature like placeholders wasn't essential for the feasibility test, I should have removed the code entirely to keep the project clean and focused.

7. Journal of the Implementation



Phase 1: Research & Setup, 29.10.2025 – 19.11.2025

- Researched sentence-transformers library and compatible models (all-MiniLM-L6-v2) that are small enough to run locally on a student laptop.
- Set up the Python environment and folder structure (models/, templates/)

Phase 2: Data Structure Design, 20.11.2025 – 03.01.2026

- Designed the templates.json schema to handle multi-lingual fields (English/German) and keywords.
- Implemented load_templates_from_json to parse this data.
- This phase took me the longest time since the AIs could not convert the Outlook templates into the required JSON format. Implementing this required a lot of “monkey job”

Phase 3: Embedding Implementation, 04.01.2026 – 17.01.2026

- Wrote the embedding generation logic.
- Implemented the vector comparison (Dot Product) using NumPy.
- *Challenge:* Ensuring the model ran completely offline without trying to connect to HuggingFace Hub

Phase 4: CLI & Integration, 17.01.2026 – 25.01.2026

- Built the command-line interface cli_embed_ver.py.
- Added the input loop (reading standard input) and output formatting.

- *Refinement:* Tweaked the text normalization to handle email headers and signatures better

Phase 5: Tests and Documentation, 25.01.2026 – 15.02.2026

- Twenty random emails were selected from Outlook and run through the system.
- The documentation was prepared and compiled

8. Learning Sources:

Sentence-Transformers Documentation: (sbert.net) - For understanding how to implement semantic search and load local models.

HuggingFace Models: For selecting the all-MiniLM-L6-v2 model based on benchmarks (speed vs. accuracy).

Argparse Tutorial - Python 2023 (Creating Your First CLI)

Vector Search RAG Tutorial (FreeCodeCamp)

Embeddings, Classification, and Semantic Search (youtube)

Sentence Transformers - EXPLAINED! (youtube)

HuggingFace + Langchain | Run 1,000s of FREE AI Models Locally (youtube)